

Docker & Kubernetes — The Complete Guide

Real-Time Deployment for Java Full Stack Developers (Spring Boot + React)

Traditional Servers → VMs → Containers → Docker → Kubernetes → Azure (AKS + Azure DevOps)

Commands • Dockerfile Internals • Security • Monitoring • Microservices • Interview Q&A

A practical, simple-language reference for interview preparation and daily real-world use

Table of Contents

1. Before Docker — Traditional Servers, VMs and Their Problems
2. Why Docker Came — The Real-World Story
3. Docker Core Concepts (Image, Container, Registry, Layers)
4. Docker Architecture — How It Works Internally
5. Installing & Setting Up Docker (Real Steps)
6. Dockerfile Deep Dive — Every Instruction Explained
7. How Many Images Can You Create From One Dockerfile? (Tags, Layers, Multi-Stage)
8. Dockerfile for Spring Boot — Full Real Example
9. Dockerfile for React — Full Real Example (Multi-Stage with Nginx)
10. Docker Compose — Running Microservices Together
11. Docker Networking & Volumes Explained
12. Docker Commands — Full Reference with What Each One Does
13. Docker Security — Before Docker vs After Docker
14. Common Docker Issues & Real-Time Fixes
15. Docker Real-World Organization Scenario (End to End)
16. Introduction to Kubernetes — Why We Needed It After Docker
17. Kubernetes Architecture — Master & Worker Nodes Explained
18. Core Kubernetes Objects (Pod, ReplicaSet, Deployment, Service, Ingress)
19. ConfigMaps, Secrets, Namespaces Explained
20. Kubernetes YAML — Full Real Examples (Spring Boot + React)
21. Kubernetes Commands (kubectl) — Full Reference
22. Monitoring & Observability Tools for Pods (Rancher, Prometheus, Grafana, Lens, K9s, Datadog)
23. Kubernetes Autoscaling & Self-Healing
24. Kubernetes Security — Before K8s vs After K8s
25. Common Kubernetes Issues & Real-Time Fixes
26. Microservices on Kubernetes — Real Organization Setup
27. Azure Integration — AKS, ACR and Azure DevOps Pipeline
28. End-to-End Real-Time Implementation Flow (Code → Prod)
29. Docker vs Kubernetes vs Other Tools — Comparison
30. Interview Questions & Answers (Basic → Advanced → Scenario Based)
31. Final Summary & Cheat Sheet

1. Before Docker — Traditional Servers, VMs and Their Problems

1.1 Era 1 — Physical/Traditional Servers

Before virtualization, every application ran on its own physical machine bought, racked, and cabled in a data center. If you had 5 applications, you bought and maintained 5 physical servers.

Traditional Physical Server Era

Physical Server 1
App A + full OS

Physical Server 2
App B + full OS

Physical Server 3
App C + full OS

- **Problem 1 — Massive waste:** each server used maybe 10-15% of its CPU/RAM capacity, but you still paid for 100% of the hardware.
- **Problem 2 — Slow provisioning:** ordering, racking, and configuring a new physical server took days to weeks.
- **Problem 3 — No isolation flexibility:** one app crashing the OS could take the whole physical box down.
- **Problem 4 — Scaling was physical:** "scale up" meant literally buying more hardware.

1.2 Era 2 — Virtual Machines (VMware, Hyper-V, KVM)

Virtualization let one physical server run multiple **Virtual Machines**, each with its own full guest OS, via a hypervisor. This solved the "one app per box" waste problem.

Virtual Machine Era

Physical Server (Host)

↓ Hypervisor (VMware/Hyper-V)

VM 1
Guest OS + App A

VM 2
Guest OS + App B

VM 3
Guest OS + App C

- **Better:** much higher hardware utilization, isolated environments, snapshots/backups became easy.
- **Still a Problem 1 — Heavy:** every VM carries a **full guest operating system** (kernel + libraries), often several GB, and takes minutes to boot.
- **Still a Problem 2 — Resource overhead:** running 10 VMs means 10 full OS kernels consuming CPU/RAM just for the OS itself, not the app.
- **Still a Problem 3 — "Works on my machine":** a developer's laptop, the QA VM, and the production VM often had subtly different OS patches, library versions, or configs — causing bugs that only appeared in one environment.
- **Still a Problem 4 — Slow CI/CD:** spinning up a new VM for testing was slow (minutes), which slowed down automated pipelines.

1.3 What Java/Spring Boot Teams Actually Faced (Real Story)

A typical pre-container Java shop had: a shared Tomcat server with multiple WARs deployed on it, an ops team manually copying WAR files via FTP/SCP, an `ops-checklist.docx` describing 20 manual steps to prepare a new environment, and constant "which JDK/library version is on this server?" confusion between Dev, QA and Prod.

Real scenario: "Our order-service used Java 8 update 202 in production, but a new VM provisioned for a scale-out event had Java 8 update 265 installed by default — a subtle date-parsing bug that only reproduced on the newer patch version caused an outage. This was purely an environment inconsistency problem — the exact class of issue containers were built to solve."

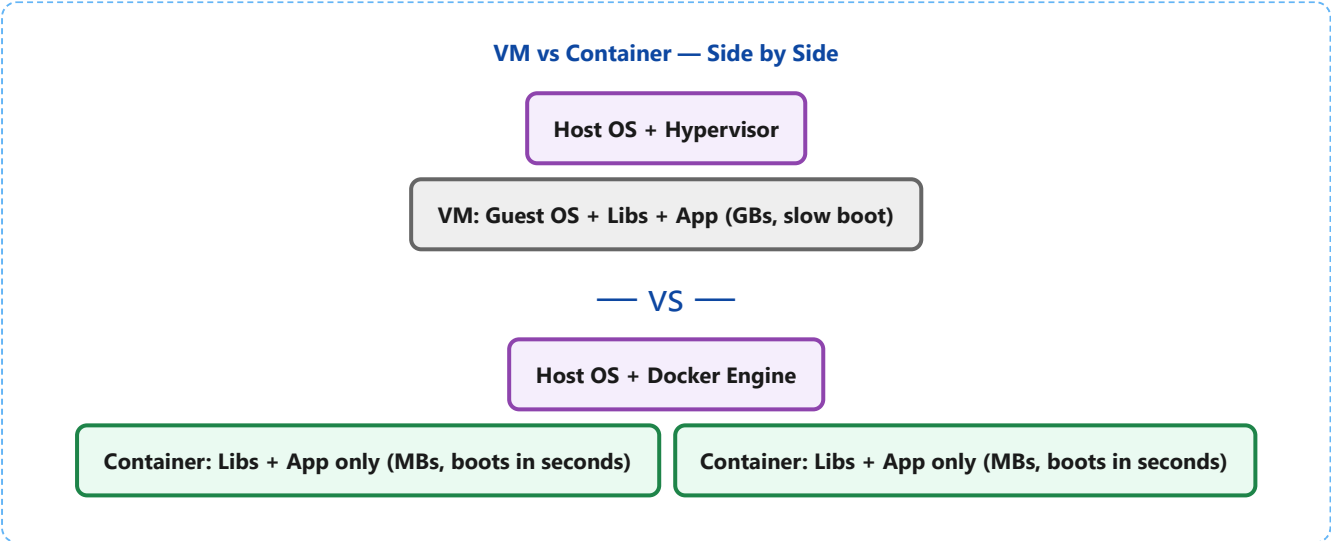
1.4 Summary of Problems Before Docker

Problem	Physical Servers	Virtual Machines
Resource utilization	Very poor	Better, but each VM still carries a full OS
Provisioning speed	Days/weeks	Minutes, still slow for CI/CD
Environment consistency	Poor	Still inconsistent — OS patches/config drift
Portability	None	Limited — VM images are huge, hypervisor-specific
Startup time	Minutes (boot server)	Minutes (boot guest OS)
Density (apps per host)	1 app per server	Several VMs per host, but overhead-heavy

2. Why Docker Came — The Real-World Story

2.1 The Core Idea Docker Introduced (2013)

Instead of virtualizing an entire computer (like a VM), Docker virtualizes at the **operating system level** — many containers share the **same host OS kernel**, but each is isolated with its own filesystem, process space, and network. No guest OS needed per app.



2.2 What Problems Docker Actually Solved

Old Problem	How Docker Solves It
"Works on my machine" bugs	The exact same image (app + dependencies + runtime) runs identically on laptop, QA, and Prod
Heavy VMs, slow boot	Containers share the host kernel — start in seconds, images are typically MBs not GBs
Manual environment setup	Dockerfile codifies the entire environment as version-controlled text
Low server density	You can run dozens of containers on a host that could only fit a few VMs
Slow CI/CD feedback	Spinning up a container for a test job takes seconds, enabling fast pipelines
Dependency hell (conflicting library versions)	Each container has its own isolated filesystem/dependencies, no clashes between apps on the same host

2.3 From a Spring Boot Perspective

Spring Boot already bundles an embedded Tomcat/Jetty into a single executable JAR — this made it a perfect fit for containers: `java -jar app.jar` inside a small JRE-based image is basically the entire deployment story. No external application server to install/configure on the host anymore.

2.4 From a React Perspective

React builds into static HTML/CSS/JS files. Before Docker, these were manually copied to a web server (Apache/Nginx/IIS) with environment-specific config. With Docker, the build output + a minimal Nginx server are packaged into one small image that behaves identically everywhere.

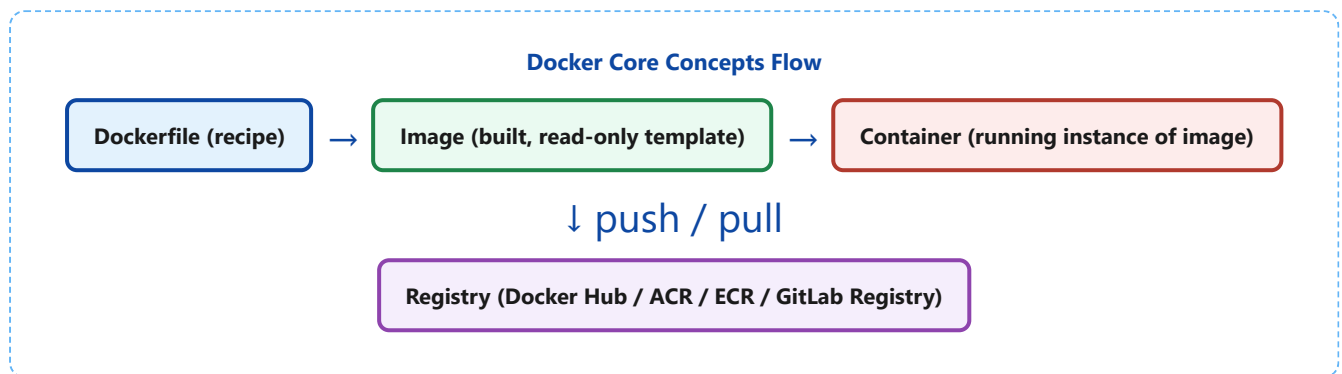
2.5 The Container Philosophy (Simple Terms)

- **Build once, run anywhere** — a Docker image is a single portable unit.
- **Immutable artifacts** — you don't patch a running container, you replace it with a new image version.
- **Declarative environment** — the Dockerfile is the single source of truth for what's inside.
- **Process isolation, not full OS duplication** — lightweight compared to VMs.

Interview one-liner: "Docker solves environment inconsistency and heavy resource usage by packaging an app with its exact dependencies into a lightweight, portable container that shares the host OS kernel instead of duplicating a full guest OS like a VM does."

3. Docker Core Concepts (Image, Container, Registry, Layers)

3.1 The Four Building Blocks



3.2 Dockerfile

A plain text file with step-by-step instructions to build an image — base OS/runtime, files to copy, dependencies to install, command to run.

3.3 Image

The built, immutable, read-only template made of stacked **layers**. An image is not running — it's a snapshot ready to be started as a container. Identified by `name:tag`, e.g., `order-service:1.4.2`.

3.4 Container

A running (or stopped) instance of an image, with a thin writable layer on top for runtime changes. You can run many containers from the same image, each isolated from the others.

3.5 Registry

A storage/distribution service for images. Examples: Docker Hub (public), Azure Container Registry (ACR), Amazon ECR, GitLab Container Registry, self-hosted Harbor/Nexus.

3.6 Image Layers Explained

Every instruction in a Dockerfile (`FROM` , `RUN` , `COPY` , etc.) creates a new, cached **layer**. Layers are stacked and shared across images — if two images both start `FROM eclipse-temurin:17-jre` , that base layer is downloaded/stored only once on a host.

Image Layer Stack (Spring Boot Example)

Layer 1: Base OS (Alpine Linux)

Layer 2: JRE 17 runtime

Layer 3: WORKDIR + app user setup

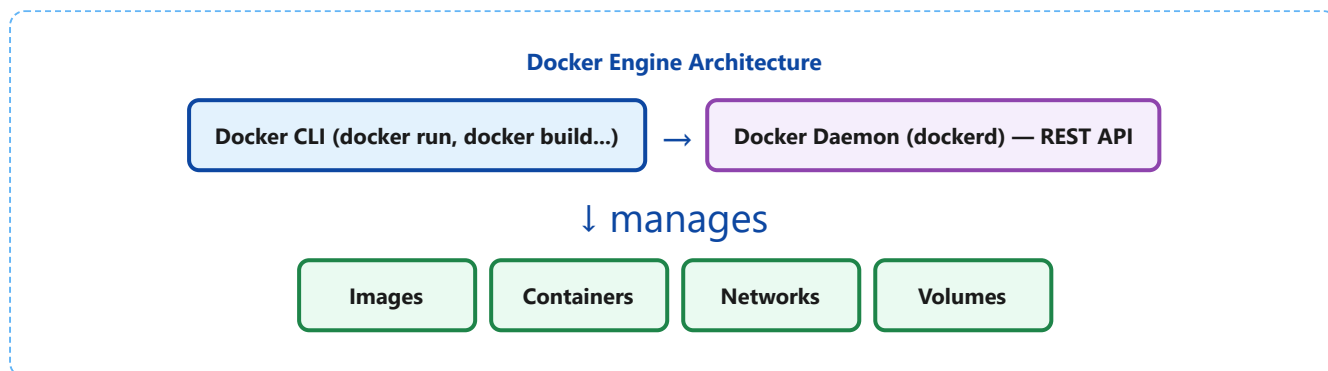
Layer 4: COPY app.jar

Thin writable layer (only exists once container runs)

Why this matters: Docker caches each layer. If you only change your application code (last **COPY** layer) and not your dependencies, rebuilding reuses all earlier cached layers — massively speeding up builds. This is why Dockerfile instruction **order** matters (see Section 6).

4. Docker Architecture — How It Works Internally

4.1 Client-Server Architecture



- **Docker CLI:** what you type (`docker build` , `docker run`) — sends commands to the daemon.
- **Docker Daemon (dockerd):** the background service that actually builds images, runs containers, manages networking/storage.
- **containerd + runc:** lower-level components that daemon delegates to for actually creating and running containers using Linux kernel features (namespaces, cgroups).

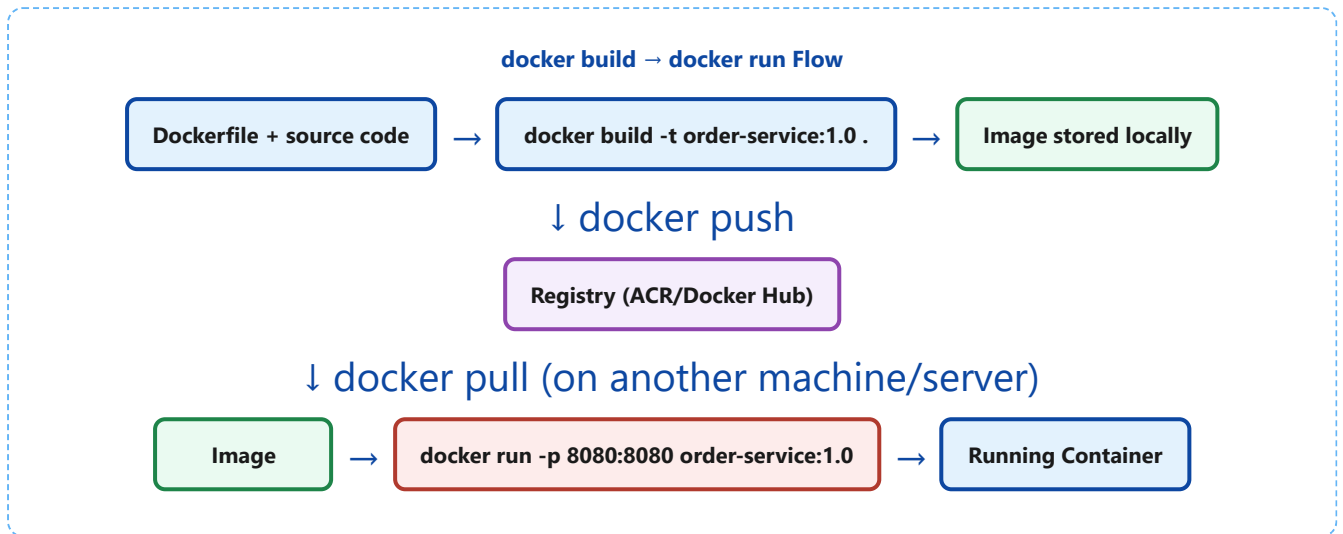
4.2 The Linux Kernel Features That Make Containers Possible

Kernel Feature	What It Provides
Namespaces	Isolation — each container gets its own view of process IDs, network interfaces, mount points, hostname
cgroups (control groups)	Resource limiting — caps CPU, memory, disk I/O per container
Union/Overlay filesystem	Layered image storage — stacks read-only layers with one writable layer on top

4.3 Docker on Windows/Mac (How It Really Runs)

Docker containers need a Linux kernel. On Windows/Mac, Docker Desktop runs a lightweight Linux VM (via WSL2 on Windows, or a hypervisor on Mac) under the hood, and your containers actually run inside that VM — this is why Docker Desktop asks you to enable WSL2/Hyper-V.

4.4 Build → Run Flow End to End



5. Installing & Setting Up Docker (Real Steps)

5.1 Local Developer Machine Setup

1. Install **Docker Desktop** (Windows/Mac) or **Docker Engine** directly (Linux).
2. On Windows: enable WSL2 backend for better performance than the legacy Hyper-V backend.
3. Verify installation: `docker --version` and `docker run hello-world`.
4. Log in to your organization's registry: `docker login myregistry.azurecr.io`.

5.2 Real Organization Server/Build-Agent Setup

- CI/CD build agents (Jenkins agent, Azure Pipelines self-hosted agent) have Docker Engine pre-installed so pipeline steps can run `docker build` / `docker push`.
- Production Kubernetes nodes use a container runtime (containerd, historically Docker Engine, now often **containerd** directly per Kubernetes' removal of "dockershim").
- Registries are private (ACR/ECR/Harbor) with role-based access — only CI/CD service principals and admins can push.

5.3 First Commands to Sanity-Check Everything

```
docker --version      # confirms Docker CLI is installed
docker info           # shows daemon status, storage driver, resource info
docker run hello-world # pulls a tiny test image and confirms daemon works end-to-end
docker images          # lists locally stored images
docker ps             # lists running containers
```

6. Dockerfile Deep Dive — Every Instruction Explained

6.1 A Complete Annotated Dockerfile

```
# Stage 1: Build (only needed to compile, not shipped in final image)
FROM maven:3.9-eclipse-temurin-17 AS build      # base image with Maven + JDK 17
WORKDIR /app                                  # sets working directory inside the image
COPY pom.xml .                                # copy pom.xml first (for dependency layer caching)
RUN mvn dependency:go-offline                  # pre-download deps – cached unless pom.xml changes
COPY src ./src                                # now copy source code (changes most often)
RUN mvn clean package -DskipTests              # compile and package into a JAR

# Stage 2: Runtime (this is what actually ships to production – small!)
FROM eclipse-temurin:17-jre-alpine             # minimal JRE-only base image (no Maven, no JDK)
RUN addgroup -S app && adduser -S app -G app     # create a non-root user (security best practice)
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar     # copy only the built JAR from Stage 1
USER app                                         # run container as non-root
EXPOSE 8080                                     # documents the port the app listens on
ENV SPRING_PROFILES_ACTIVE=prod                 # default environment variable
HEALTHCHECK --interval=30s --timeout=3s CMD wget -qO- http://localhost:8080/actuator/health || exit 1
ENTRYPOINT ["java", "-jar", "app.jar"]          # fixed command that always runs
```

6.2 Every Instruction Explained

Instruction	What It Does
FROM	Chooses the base image to start from — every Dockerfile must start with this (or ARG before it)
WORKDIR	Sets/creates the working directory for subsequent instructions — avoids messy absolute paths
COPY	Copies files from build context (your machine/CI) into the image
ADD	Like COPY but also supports remote URLs and auto-extracting tar archives — generally COPY is preferred unless you need those extras
RUN	Executes a command at build time , result is committed as a new layer (e.g., installing packages, compiling code)
ENV	Sets an environment variable available at build and container run time
ARG	A build-time-only variable (not available in the running container unless also set as ENV)
EXPOSE	Documents which port the container listens on (does not actually publish it — that's -p on docker run)
USER	Switches the user the rest of the instructions/container run as — critical for security (avoid root)
CMD	Default command/arguments if none given at docker run — can be overridden
ENTRYPOINT	The fixed main command that always runs — harder to override, often combined with CMD for default args
HEALTHCHECK	Tells Docker how to test if the container is actually healthy, not just "running"
VOLUME	Marks a directory as a mount point for persistent/external storage
LABEL	Adds metadata (maintainer, version, git commit hash) to the image

6.3 CMD vs ENTRYPOINT — The Classic Confusion

	CMD	ENTRYPOINT
Overridable?	Yes, easily replaced by <code>docker run image other-command</code>	No (unless <code>--entrypoint</code> flag used) — meant to be fixed
Typical use	Default arguments	The actual program that must always run
Best pattern	Combine both: <code>ENTRYPOINT ["java", "-jar", "app.jar"]</code> with optional <code>CMD ["--spring.profiles.active=dev"]</code> as default overridable args	

6.4 Dockerfile Instruction Order Matters (Build Speed)

Put things that change **least often** (base image, dependency manifests) at the **top**, and things that change **most often** (your source code) at the **bottom**. This maximizes Docker's layer cache reuse and dramatically speeds up rebuilds — this is why `COPY pom.xml` happens before `COPY src` in the example above.

6.5 .dockerignore — Don't Forget This

```
# .dockerignore – keeps build context small and avoids leaking secrets/junk into images
target/
node_modules/
.git/
*.log
.env
```

7. How Many Images Can You Create From One Dockerfile? (Tags, Layers, Multi-Stage)

7.1 One Dockerfile → Unlimited Images (via Tags)

A Dockerfile is just a build recipe. Every time you run `docker build`, you can tag the result differently — so a single Dockerfile can produce as many distinct, independently stored images as you want, each identified by its own `name:tag`.

```
docker build -t order-service:1.0.0 .
docker build -t order-service:1.0.1 .
docker build -t order-service:latest .
docker build --build-arg APP_ENV=qa -t order-service:qa-1.0.0 .
```

- Same Dockerfile + same source code, built twice → produces two images with **identical layer hashes** (Docker recognizes no change occurred, near-instant "build" due to caching).
- Same Dockerfile + different source code/args each build → each build produces a genuinely new image with a new content hash, even under the same tag name (old tag reference simply moves to the new image).

7.2 Multi-Stage Builds — Multiple "Images" in One Dockerfile

A single Dockerfile can define multiple named `FROM ... AS stageName` stages. Each stage technically builds its own intermediate image, but only the **final stage** becomes the image you actually ship — earlier stages (compilers, build tools) are discarded, keeping the shipped image small and secure.

```
FROM node:20 AS build          # Stage 1 – has Node, npm, dev dependencies (large)
...
FROM nginx:alpine AS final      # Stage 2 – final shipped image (small, no Node at all)
COPY --from=build /app/dist /usr/share/nginx/html
```

- You can even build and tag **intermediate stages** separately for debugging: `docker build --target build -t order-service:build-debug .`
- Real orgs commonly keep 3 tags per release: the immutable `<build-number>` tag, a semantic version tag like `1.4.2`, and a moving `latest` / `stable` tag.

7.3 Tagging Strategy Used in Real Organizations

Tag Style	Example	Purpose
Build number	<code>order-service:482</code>	Immutable, traceable to exact CI/CD pipeline run
Git SHA	<code>order-service:a1b2c3d</code>	Traceable to exact commit, useful for debugging "what code is actually running"
Semantic version	<code>order-service:2.3.0</code>	Human-readable release version, follows semver
Environment moving tag	<code>order-service:qa-latest</code>	Always points to whatever is currently deployed in QA

latest	order-service:latest	Avoid relying on this in production — ambiguous, not reproducible
--------	----------------------	---

Interview tip: Never deploy production using the `latest` tag — it's a moving target and you can't be 100% sure which exact build is running. Always deploy an immutable tag (build number or git SHA).

8. Dockerfile for Spring Boot — Full Real Example

8.1 Project Structure

```
order-service/
├─ src/main/java/...
├─ src/main/resources/application.yml
├─ pom.xml
├─ Dockerfile
└─ .dockerignore
```

8.2 Production-Grade Multi-Stage Dockerfile

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -B
COPY src ./src
RUN mvn clean package -DskipTests -B

FROM eclipse-temurin:17-jre-alpine
RUN addgroup -S spring && adduser -S spring -G spring
WORKDIR /app
COPY --from=build /app/target/order-service-*.jar app.jar
RUN chown spring:spring app.jar
USER spring
EXPOSE 8080
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
  CMD wget -qO- http://localhost:8080/actuator/health | grep UP || exit 1
ENTRYPOINT ["java", "-XX:+UseContainerSupport", "-XX:MaxRAMPercentage=75.0", "-jar", "app.jar"]
```

Why `-XX:+UseContainerSupport` and `MaxRAMPercentage` ? Without these JVM flags, older JVMs could misread the **host's** total memory instead of the container's cgroup memory limit, causing OOM-killed containers even when heap usage looked "fine" from the JVM's own perspective. Modern JDKs (10+) detect cgroup limits automatically, but explicitly setting `MaxRAMPercentage` is still a common real-world safeguard.

8.3 Building & Running It

```
docker build -t order-service:1.0.0 .
docker run -d -p 8080:8080 --name order-service \
  -e SPRING_PROFILES_ACTIVE=prod \
  -e DB_URL=jdbc:postgresql://prod-db:5432/orders \
  order-service:1.0.0
```

8.4 Image Size Comparison (Real Numbers)

Base Image Choice	Approx. Final Image Size	Notes
-------------------	--------------------------	-------

<code>ubuntu</code> + manual JDK install	~450-600 MB	Avoid — large, more attack surface
<code>eclipse-temurin:17-jdk</code>	~280 MB	Full JDK not needed at runtime
<code>eclipse-temurin:17-jre-alpine</code>	~120-150 MB	Good real-world default for Spring Boot
Distroless Java base	~100 MB, no shell at all	Best security posture, harder to debug interactively

9. Dockerfile for React — Full Real Example (Multi-Stage with Nginx)

9.1 Project Structure

```
customer-portal/  
├─ src/  
├─ public/  
├─ package.json  
├─ .env.production  
├─ Dockerfile  
├─ nginx.conf  
└─ .dockerignore
```

9.2 Multi-Stage Dockerfile

```
# Stage 1: Build the static bundle  
FROM node:20-alpine AS build  
WORKDIR /app  
COPY package.json package-lock.json ./  
RUN npm ci # reproducible install using the lock file  
COPY . .  
RUN npm run build # produces /app/build (or /dist for Vite)  
  
# Stage 2: Serve with Nginx – tiny final image, no Node.js at all  
FROM nginx:1.27-alpine  
COPY --from=build /app/build /usr/share/nginx/html  
COPY nginx.conf /etc/nginx/conf.d/default.conf  
EXPOSE 80  
HEALTHCHECK --interval=30s CMD wget -qO- http://localhost:80 || exit 1  
CMD ["nginx", "-g", "daemon off;"]
```

9.3 nginx.conf (Handles React Router + API Proxy)

```
server {  
    listen 80;  
    root /usr/share/nginx/html;  
    index index.html;  
  
    location / {  
        try_files $uri /index.html; # React Router – always fall back to index.html  
    }  
  
    location /api/ {  
        proxy_pass http://order-service:8080/; # proxy calls to backend service inside the cluster  
    }  
}
```

9.4 Environment Config for React (Build-Time vs Runtime)

React environment variables (`REACT_APP_*` / Vite's `VITE_*`) are baked into the static bundle **at build time** — they cannot be changed after the image is built like a backend env var can. Real orgs solve this by either (a) building one image per environment, or (b) injecting a small `env-config.js` file at container startup that the app reads at runtime.

```
# Runtime-injectable config approach – entrypoint script generates env-config.js from container env vars
#!/bin/sh
echo "window._env_ = { API_URL: \"\$API_URL\" };" > /usr/share/nginx/html/env-config.js
exec nginx -g 'daemon off;'
```

9.5 Building & Running It

```
docker build -t customer-portal:1.0.0 .
docker run -d -p 80:80 --name customer-portal customer-portal:1.0.0
```

10. Docker Compose — Running Microservices Together

10.1 Why Compose Exists

A real application is rarely one container — it's a Spring Boot backend + a Postgres database + Redis cache + a React frontend, all needing to talk to each other. Docker Compose lets you define this whole multi-container stack in one YAML file and start it with a single command — perfect for local development and demos.

10.2 Real docker-compose.yml — Microservices Example

```
version: "3.9"
services:
  order-service:
    build: ./order-service
    image: order-service:local
    ports: [ "8080:8080" ]
    environment:
      SPRING_PROFILES_ACTIVE: dev
      DB_URL: jdbc:postgresql://postgres:5432/orders
    depends_on:
      postgres: { condition: service_healthy }
    networks: [ backend-net ]

  customer-portal:
    build: ./customer-portal
    image: customer-portal:local
    ports: [ "80:80" ]
    depends_on: [ order-service ]
    networks: [ backend-net ]

  postgres:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: orders
      POSTGRES_PASSWORD: devpass
    volumes: [ "pgdata:/var/lib/postgresql/data" ]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      retries: 5
    networks: [ backend-net ]

  redis:
    image: redis:7-alpine
    networks: [ backend-net ]

volumes:
  pgdata:

networks:
  backend-net:
```

10.3 Key Commands

```
docker compose up -d          # builds (if needed) and starts the whole stack in background
docker compose logs -f order-service  # tail logs of one service
docker compose down          # stops and removes containers/networks (add -v to also remove volumes)
docker compose ps            # status of all services in the stack
```

Real scenario: "New developers onboarding to our team run `docker compose up` once and get the full backend + database + cache + frontend running locally in under 2 minutes — no manual Postgres install, no version mismatch, no 20-step setup doc."

11. Docker Networking & Volumes Explained

11.1 Docker Networking Modes

Network Type	Behavior	Typical Use
Bridge (default)	Private internal network on the host; containers reach each other by name	Local dev, Compose stacks
Host	Container shares the host's network namespace directly (no port mapping needed)	Performance-sensitive cases, rare in prod
None	No networking at all	Fully isolated batch jobs
Overlay	Spans multiple Docker hosts (Swarm) — conceptually similar to Kubernetes' pod networking	Multi-host clustering

Compose Bridge Network — Service Discovery by Name



11.2 Volumes — Persisting Data Beyond the Container's Life

Containers are ephemeral by design — delete one and its writable layer is gone. A **Volume** is storage managed by Docker outside the container's lifecycle, so data (like a database's files) survives container restarts/recreation.

Storage Type	Managed By	Survives container removal?	Typical Use
Named Volume	Docker	Yes	Database data directories
Bind Mount	You (maps a host path directly)	Yes (it's just a host folder)	Local dev — live-reload source code into a container
tmpfs Mount	Docker (in-memory)	No — gone on stop	Sensitive temp data, caches

```
docker volume create pgdata
docker run -d -v pgdata:/var/lib/postgresql/data postgres:15-alpine
docker run -d -v $(pwd)/src:/app/src order-service:dev # bind mount for live dev
```

12. Docker Commands — Full Reference with What Each One Does

12.1 Image Commands

Command	What It Does
<code>docker build -t name:tag .</code>	Builds an image from a Dockerfile in the current directory
<code>docker images</code>	Lists all locally stored images
<code>docker pull image:tag</code>	Downloads an image from a registry
<code>docker push image:tag</code>	Uploads an image to a registry (must be logged in and tagged correctly)
<code>docker tag src:tag new:tag</code>	Creates an additional tag pointing to the same image
<code>docker rmi image:tag</code>	Removes a local image
<code>docker history image:tag</code>	Shows each layer and its size — useful for finding what's bloating an image
<code>docker image prune -a</code>	Removes all unused images to free disk space

12.2 Container Commands

Command	What It Does
<code>docker run -d -p 8080:8080 image</code>	Starts a new container in detached mode, mapping host port 8080 to container port 8080
<code>docker ps</code>	Lists running containers
<code>docker ps -a</code>	Lists all containers including stopped ones
<code>docker stop container</code>	Gracefully stops a running container (sends SIGTERM, then SIGKILL after timeout)
<code>docker start container</code>	Restarts a previously stopped container
<code>docker restart container</code>	Stops then starts a container
<code>docker rm container</code>	Deletes a stopped container
<code>docker logs -f container</code>	Streams the container's stdout/stderr logs live
<code>docker exec -it container sh</code>	Opens an interactive shell inside a running container — key debugging tool
<code>docker inspect container</code>	Dumps full JSON metadata — IP address, mounts, env vars, health status
<code>docker stats</code>	Live CPU/memory/network usage per running container
<code>docker cp container:/path ./local</code>	Copies a file out of (or into) a container
<code>docker top container</code>	Shows running processes inside a container

12.3 System & Cleanup Commands

Command	What It Does
<code>docker system df</code>	Shows disk space used by images, containers, volumes, cache
<code>docker system prune -a --volumes</code>	Deep clean — removes all unused containers, images, networks, and volumes (use carefully!)
<code>docker network ls</code>	Lists networks
<code>docker volume ls</code>	Lists volumes

12.4 Real-Time Debugging Scenario Using Commands

Real scenario: "Container keeps restarting" — first `docker ps -a` to see exit code, then `docker logs container_id` to see the crash reason, then `docker inspect container_id` to check memory limits (an exit code 137 usually means OOM-killed), then reproduce interactively with `docker run -it --entrypoint sh image` to poke around before the crash-looping entrypoint runs.

13. Docker Security — Before Docker vs After Docker

13.1 Security Before Docker (VM/Physical Era)

- Isolation was at the OS/hypervisor level — strong, but coarse-grained (one whole VM per workload).
- Patch management meant patching whole guest OS images per VM — slow, inconsistent across teams.
- Attack surface = entire OS + all installed packages, even ones the app didn't need.
- Secrets were often placed in server-level config files or environment scripts, sometimes checked into shared file shares.

13.2 Security Concerns Docker Introduced

Because containers share the host kernel, a container escape vulnerability is more impactful than a VM escape (rarer, but the blast radius includes the whole host). Also, most base images historically defaulted to running as **root** inside the container, which is risky if an app is compromised.

13.3 Docker Security Best Practices (After Docker, Done Right)

Practice	Why
Run as non-root (<code>USER app</code>)	Limits what an attacker can do if the app process is compromised
Use minimal base images (Alpine/Distroless)	Smaller attack surface — fewer packages, fewer CVEs
Multi-stage builds	Build tools/compilers never ship in the final image
Scan images (Trivy, Snyk, Docker Scout, ACR/ECR built-in scanning)	Finds known CVEs in base image + dependencies before deployment
Sign images (Docker Content Trust / Cosign)	Verifies the image wasn't tampered with between build and deploy
Never bake secrets into images	Anyone who pulls the image can extract layer contents — secrets must come from Key Vault/Secrets Manager at runtime
Read-only root filesystem (<code>--read-only</code>)	Prevents malware/tampering from writing to the container filesystem at runtime
Drop unnecessary Linux capabilities (<code>--cap-drop=ALL</code>)	Reduces what a compromised process could exploit at the kernel level
Resource limits (<code>--memory</code> , <code>--cpus</code>)	Prevents one compromised/runaway container from starving the whole host
Private registries with RBAC	Prevents unauthorized push/pull of production images

13.4 Real Image Scanning in a Pipeline

```
# Trivy scan step added to CI before pushing to registry
trivy image --severity HIGH,CRITICAL --exit-code 1 order-service:1.0.0
# pipeline fails the build if HIGH/CRITICAL vulnerabilities are found – blocks insecure images from shipping
```

13.5 Summary Table — Before vs After

Aspect	Before Docker	After Docker (Done Right)
Isolation granularity	Whole VM/server	Per-process container, but shares host kernel
Patch scope	Whole guest OS	Rebuild + redeploy small image layers
Default user	Varies by server config	Should always be explicit non-root user
Vulnerability visibility	Manual OS-level audits, infrequent	Automated image scanning on every build
Secret handling	Config files on shared servers	Injected at runtime from Key Vault/Secrets Manager, never in image layers

14. Common Docker Issues & Real-Time Fixes

Dockerfile Instruction Order — Why It Controls Build Speed

Bad: COPY . . → RUN npm install (every code change invalidates the install layer)

Good: COPY package.json → RUN npm install → COPY . . (install layer cached until package.json changes)

Issue	Cause	Real Fix
Container exits immediately after start	Main process (ENTRYPOINT/CMD) finished or crashed instantly	<code>docker logs</code> to see the error; ensure the main process runs in the foreground (not backgrounded/daemonized)
"Port already in use" on <code>docker run</code>	Host port already bound by another process/container	Use a different host port mapping (<code>-p 8081:8080</code>) or stop the conflicting container
Image is huge (GBs)	Not using multi-stage build, wrong base image, leftover build cache/tools in final layer	Multi-stage build, minimal base image (Alpine/Distroless), clean apt/npm caches in same RUN layer
Every code change rebuilds everything from scratch (slow)	Wrong instruction order — source code copied before dependency manifest	Reorder Dockerfile: copy dependency files first, install deps, then copy source
Container OOM-killed (exit code 137)	No/incorrect memory limits, JVM not respecting container cgroup limits	Set <code>--memory</code> limit matching real usage, use <code>-XX:MaxRAMPercentage</code> for JVM apps, profile actual heap needs
"Cannot connect to the Docker daemon"	Docker Desktop/daemon not running, or permission issue (non-root user not in <code>docker</code> group on Linux)	Start Docker Desktop/daemon; on Linux add user to docker group or use sudo appropriately
Container can't reach another container by name	Containers on different Docker networks, or not using Compose's automatic network	Ensure both are on the same user-defined network; avoid relying on default bridge for service discovery
Secrets visible in <code>docker history</code> /image layers	Secret was passed via <code>ENV</code> / <code>ARG</code> baked into a layer instead of runtime injection	Never bake secrets in build; use runtime env vars/secret mounts, use <code>--secret</code> with BuildKit if a build-time secret is unavoidable
Disk fills up on build servers	Old dangling images/build cache never cleaned	Regular <code>docker system prune</code> in CI agent maintenance, or use ephemeral hosted agents
"Works on my machine" even with Docker	Different base image tag pulled (e.g., <code>latest</code> drifted), platform mismatch (ARM Mac vs x86 prod)	Pin exact base image digests, build multi-platform images (<code>docker buildx</code>) if dev/prod architectures differ
Container healthy locally but fails health check in CI/registry scan	Missing <code>HEALTHCHECK</code> instruction, or scan runs before app fully starts	Add an explicit <code>HEALTHCHECK</code> with a realistic start period; don't rely solely on process-exists checks
Build fails only on the CI agent, never locally	CI agent uses a different Docker/BuildKit version or lacks build cache warm from	Pin Docker/BuildKit version in the pipeline, enable registry-based layer caching so CI benefits from the

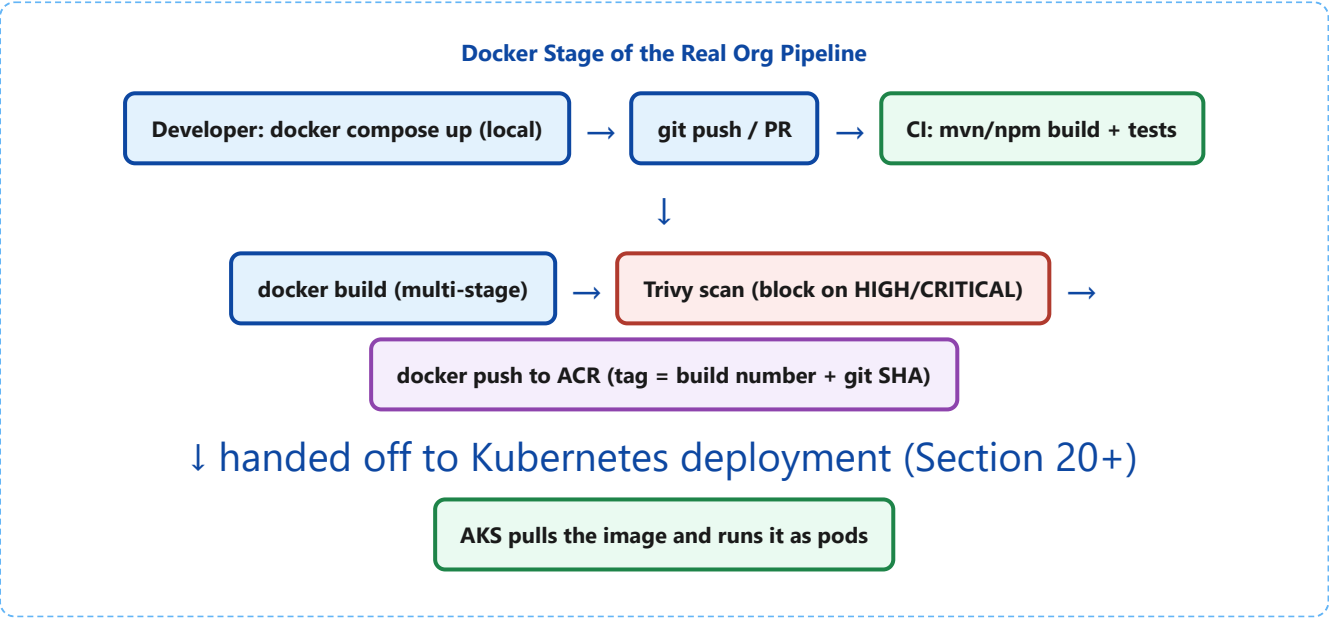
	previous runs	same cache as local builds
--	---------------	----------------------------

15. Docker Real-World Organization Scenario (End to End)

15.1 The Scenario

A mid-size fintech company runs a Spring Boot "order-service", a Spring Boot "payment-service", a React "customer-portal", Postgres, and Redis. All developers use Docker Compose locally; CI builds and scans Docker images; images are pushed to Azure Container Registry; production runs on Kubernetes (covered from Section 16 onward).

15.2 End-to-End Flow



15.3 What Each Team Owns

Team	Responsibility
Backend developers	Write/maintain each service's Dockerfile, keep image small and secure
Frontend developers	Maintain React Dockerfile + nginx.conf, manage env-config injection
DevOps/Platform team	Own the CI pipeline steps, registry access policy, scanning gate rules
Security team	Defines which CVEs block a release, reviews base image approval list

Real scenario: "When the security team mandated a switch from `eclipse-temurin:17-jdk` to `eclipse-temurin:17-jre-alpine` org-wide, every team's Dockerfile FROM line was updated via a shared base-image standard, cutting average image size by 60% and closing a batch of previously flagged CVEs in one release cycle."

16. Introduction to Kubernetes — Why We Needed It After Docker

16.1 The Problem Docker Alone Doesn't Solve

Docker is excellent at packaging and running **one** container on **one** host. But real organizations run hundreds of containers across many hosts, and need answers to: What happens when a container crashes? How do we scale to 10 replicas during peak traffic? How do containers on different hosts find each other? How do we roll out updates without downtime? Docker alone doesn't answer these — that's what **Kubernetes (K8s)** (originally built by Google, based on their internal "Borg" system) was built for.

16.2 What Kubernetes Actually Does

Need	What Kubernetes Provides
Container crashes	Automatically restarts it (self-healing)
Traffic spikes	Horizontal Pod Autoscaler adds more replicas automatically
Multi-host networking	Built-in cluster networking so pods find each other by name regardless of which node they're on
Zero-downtime updates	Rolling updates, readiness probes control when new pods receive traffic
Node failure	Reschedules pods from a dead node onto healthy nodes automatically
Config/secret management at scale	ConfigMaps and Secrets decoupled from container images
Declarative infrastructure	You describe the desired state (YAML); Kubernetes continuously works to match reality to it

16.3 Kubernetes vs Plain Docker — Side by Side

Docker Alone vs Docker + Kubernetes

Docker alone: you manually run/restart/scale containers on each host

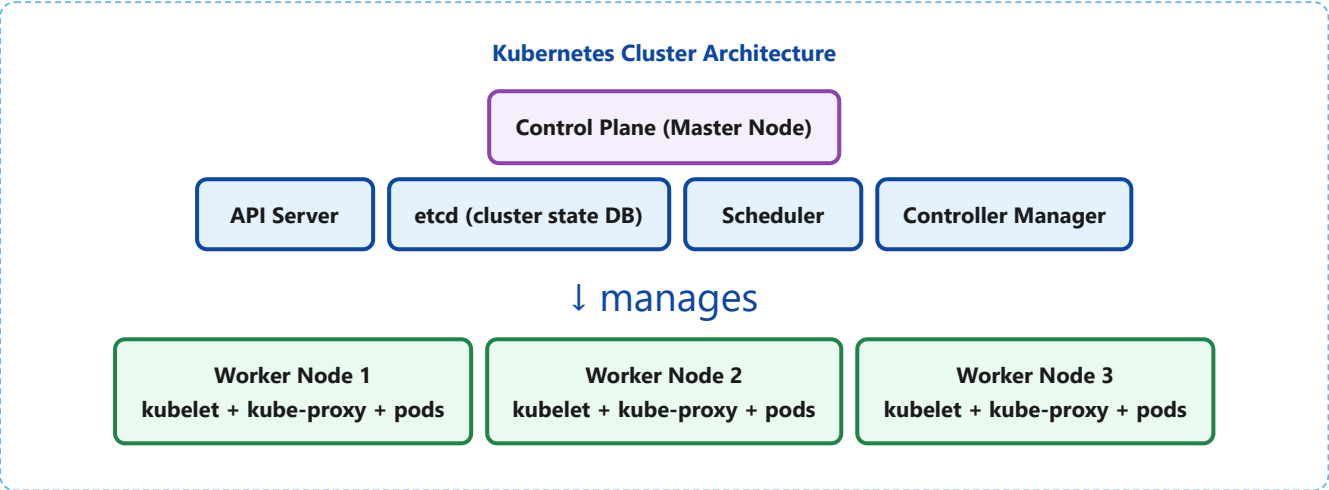
— VS —

Kubernetes: you declare "I want 4 replicas of order-service, always healthy" — K8s makes it true, continuously

Interview one-liner: "Docker packages and runs a single container; Kubernetes orchestrates many containers across many machines — handling scheduling, scaling, networking, self-healing, and rollouts automatically based on a declared desired state."

17. Kubernetes Architecture — Master & Worker Nodes Explained

17.1 Cluster Overview



17.2 Control Plane Components

Component	Role
API Server	The front door — every <code>kubectl</code> command and internal component talks to the cluster only through this REST API
etcd	Distributed key-value store holding the entire cluster's desired/actual state — the single source of truth
Scheduler	Decides which worker node a new pod should run on, based on resource requests/limits, affinity rules, taints
Controller Manager	Runs control loops that constantly reconcile actual state to desired state (e.g., "I need 4 replicas but only see 3 — create one more")

17.3 Worker Node Components

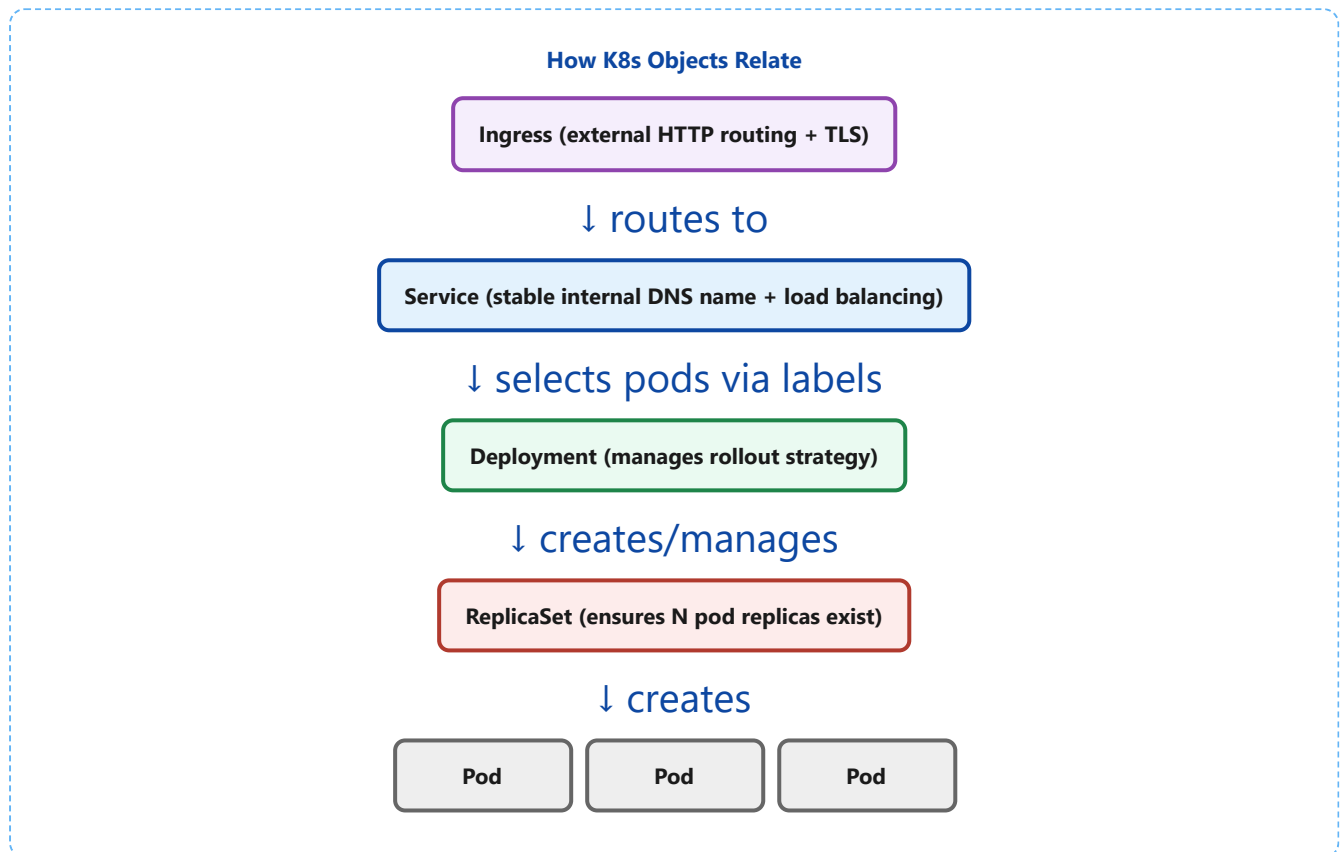
Component	Role
kubelet	Agent on each node that talks to the API server and ensures containers described for that node are actually running
Container Runtime (containerd)	Actually pulls images and runs containers (Docker Engine historically, now typically containerd directly)
kube-proxy	Manages network rules on the node so Services can route traffic to the right pods

17.4 The Reconciliation Loop (Core Kubernetes Philosophy)

Every controller in Kubernetes follows the same pattern: **Observe** current state → **Compare** to desired state (your YAML) → **Act** to close the gap → repeat forever. This is why deleting a pod managed by a Deployment doesn't actually remove it for long — the ReplicaSet controller immediately notices "3 desired, 2 actual" and creates a replacement.

18. Core Kubernetes Objects (Pod, ReplicaSet, Deployment, Service, Ingress)

18.1 The Object Hierarchy



18.2 Pod

The smallest deployable unit in Kubernetes — one or more tightly coupled containers that share network/storage (usually just 1 container per pod for microservices, e.g., one Spring Boot container).

18.3 ReplicaSet

Ensures a specified number of identical pod replicas are running at all times — replaces a crashed pod automatically. You rarely create these directly; Deployments manage them for you.

18.4 Deployment

The object you actually work with day-to-day. Manages ReplicaSets, handles rolling updates/rollbacks, and describes the desired state of your application (image version, replica count, resource limits, probes).

18.5 Service

Gives a stable network identity (a DNS name and virtual IP) to a set of pods, even as individual pods are replaced. Types:

ClusterIP (internal only, default), **NodePort** (exposes on each node's IP), **LoadBalancer** (provisions a cloud load balancer, e.g., Azure Load Balancer).

18.6 Ingress

Manages external HTTP/HTTPS access into the cluster, with host/path-based routing and TLS termination — usually backed by an Ingress Controller like NGINX Ingress or Azure Application Gateway Ingress Controller (AGIC).

18.7 StatefulSet & DaemonSet (Good to Know)

Object	Use Case
StatefulSet	For stateful apps needing stable network identity and storage per replica (e.g., a Kafka broker, self-managed database)
DaemonSet	Ensures one pod runs on every (or selected) node — used for log collectors, monitoring agents
Job / CronJob	Run-to-completion batch tasks, or scheduled recurring tasks (e.g., nightly report generation)

19. ConfigMaps, Secrets, Namespaces Explained

19.1 ConfigMap

Stores non-sensitive configuration (URLs, feature flags, log levels) separately from the container image, so the same image can run in Dev/QA/Prod with different config injected as env vars or mounted files.

```
apiVersion: v1
kind: ConfigMap
metadata: { name: order-service-config, namespace: prod }
data:
  LOG_LEVEL: "INFO"
  FEATURE_NEW_CHECKOUT: "true"
```

19.2 Secret

Same idea as ConfigMap but intended for sensitive values — base64-encoded at rest by default (not encrypted unless you enable encryption-at-rest or use an external secrets provider like Azure Key Vault via CSI driver).

```
apiVersion: v1
kind: Secret
metadata: { name: order-service-secrets, namespace: prod }
type: Opaque
data:
  DB_PASSWORD: cGFzc3dvcnQxMjM= # base64 encoded – NOT encryption, just encoding
```

Base64 is **not** encryption — anyone with API access can decode it instantly. Real organizations integrate an external secret manager (Azure Key Vault Provider for Secrets Store CSI Driver, HashiCorp Vault) so actual secret values never sit in etcd in a reversible format, and enable etcd encryption-at-rest as a baseline.

19.3 Namespaces — Logical Isolation

A Namespace partitions one physical cluster into multiple virtual clusters — commonly one per environment (`dev` , `qa` , `staging` , `prod`) or per team, with its own resource quotas and RBAC rules.

```
kubectl create namespace dev
kubectl get pods -n prod
kubectl config set-context --current --namespace=dev # switch default namespace for convenience
```

Namespace	Typical Use
kube-system	Kubernetes' own control components (DNS, kube-proxy, etc.) — don't touch
dev / qa / staging / prod	One cluster shared across environments, isolated logically (common cost-saving pattern for non-prod)
monitoring	Prometheus, Grafana, Alertmanager stack

ingress-nginx	Ingress controller components
---------------	-------------------------------

20. Kubernetes YAML — Full Real Examples (Spring Boot +

React)

20.1 Spring Boot — Deployment + Service + Ingress

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-service
  namespace: prod
  labels: { app: order-service }
spec:
  replicas: 4
  strategy:
    type: RollingUpdate
    rollingUpdate: { maxUnavailable: 1, maxSurge: 1 }
  selector:
    matchLabels: { app: order-service }
  template:
    metadata:
      labels: { app: order-service }
    spec:
      containers:
        - name: order-service
          image: myacr.azurecr.io/order-service:482
          ports: [{ containerPort: 8080 }]
          resources:
            requests: { cpu: "250m", memory: "384Mi" }
            limits: { cpu: "500m", memory: "512Mi" }
          readinessProbe:
            httpGet: { path: /actuator/health/readiness, port: 8080 }
            initialDelaySeconds: 10
            periodSeconds: 5
          livenessProbe:
            httpGet: { path: /actuator/health/liveness, port: 8080 }
            initialDelaySeconds: 20
            periodSeconds: 10
          envFrom:
            - configMapRef: { name: order-service-config }
            - secretRef: { name: order-service-secrets }
---
apiVersion: v1
kind: Service
metadata: { name: order-service, namespace: prod }
spec:
  selector: { app: order-service }
  ports: [{ port: 80, targetPort: 8080 }]
  type: ClusterIP
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: order-service-ingress
  namespace: prod
  annotations: { nginx.ingress.kubernetes.io/rewrite-target: / }
spec:
  ingressClassName: nginx
  rules:
    - host: api.mycompany.com

```

```

http:
  paths:
    - path: /orders
      pathType: Prefix
      backend:
        service: { name: order-service, port: { number: 80 } }
tls:
- hosts: [ api.mycompany.com ]
  secretName: mycompany-tls

```

20.2 React Frontend — Deployment + Service + Ingress

```

apiVersion: apps/v1
kind: Deployment
metadata: { name: customer-portal, namespace: prod }
spec:
  replicas: 3
  selector: { matchLabels: { app: customer-portal } }
  template:
    metadata: { labels: { app: customer-portal } }
    spec:
      containers:
        - name: customer-portal
          image: myacr.azurecr.io/customer-portal:212
          ports: [{ containerPort: 80 }]
          resources:
            requests: { cpu: "100m", memory: "128Mi" }
            limits: { cpu: "250m", memory: "256Mi" }
          readinessProbe:
            httpGet: { path: /, port: 80 }
            initialDelaySeconds: 5
---
apiVersion: v1
kind: Service
metadata: { name: customer-portal, namespace: prod }
spec:
  selector: { app: customer-portal }
  ports: [{ port: 80, targetPort: 80 }]
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata: { name: customer-portal-ingress, namespace: prod }
spec:
  ingressClassName: nginx
  rules:
    - host: app.mycompany.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service: { name: customer-portal, port: { number: 80 } }

```

Real scenario: "api.mycompany.com routes to backend services and app.mycompany.com routes to the React frontend — both share the same NGINX Ingress Controller and the same wildcard TLS certificate managed by cert-manager, which auto-renews from Let's Encrypt."

21. Kubernetes Commands (kubectl) — Full Reference

21.1 Cluster & Context

Command	What It Does
<code>kubectl cluster-info</code>	Shows the control plane and DNS endpoint of the connected cluster
<code>kubectl config get-contexts</code>	Lists available cluster contexts (useful with multiple clusters/environments)
<code>kubectl config use-context aks-prod</code>	Switches the active cluster context

21.2 Viewing Resources

Command	What It Does
<code>kubectl get pods -n prod</code>	Lists pods in the prod namespace
<code>kubectl get pods -o wide</code>	Adds node name and IP to pod listing
<code>kubectl get deployments,svc,ingress -n prod</code>	Lists multiple resource types at once
<code>kubectl describe pod order-service-abc123 -n prod</code>	Full details + recent events — first command to run when a pod misbehaves
<code>kubectl get events -n prod --sort-by=.lastTimestamp</code>	Chronological cluster events — great for diagnosing scheduling failures

21.3 Logs & Debugging

Command	What It Does
<code>kubectl logs order-service-abc123 -n prod</code>	Shows container logs
<code>kubectl logs -f order-service-abc123 --previous</code>	Streams logs, <code>--previous</code> shows logs from before the last crash/restart
<code>kubectl exec -it order-service-abc123 -- sh</code>	Opens an interactive shell inside the running pod's container
<code>kubectl port-forward svc/order-service 8080:80 -n prod</code>	Tunnels a local port to a service/pod inside the cluster for local debugging
<code>kubectl top pods -n prod</code>	Live CPU/memory usage per pod (needs metrics-server)

21.4 Applying & Managing Changes

Command	What It Does
<code>kubectl apply -f deployment.yml</code>	Creates or updates resources to match the YAML (declarative — the standard way)

<code>kubectl set image deployment/order-service order-service=myacr.azurecr.io/order-service:483 -n prod</code>	Updates just the image version, triggering a rolling update
<code>kubectl rollout status deployment/order-service -n prod</code>	Watches a rollout until it completes or fails
<code>kubectl rollout history deployment/order-service -n prod</code>	Lists previous rollout revisions
<code>kubectl rollout undo deployment/order-service -n prod</code>	Instantly rolls back to the previous revision
<code>kubectl scale deployment/order-service --replicas=8 -n prod</code>	Manually scales replica count
<code>kubectl delete pod order-service-abc123 -n prod</code>	Deletes a pod — its Deployment/ReplicaSet immediately creates a replacement

21.5 Real Debugging Sequence (What You Actually Type)

```
kubectl get pods -n prod # see CrashLoopBackOff / Pending / ImagePullBackOff status
kubectl describe pod order-service-abc123 -n prod # check Events section at the bottom for the real reason
kubectl logs order-service-abc123 -n prod --previous # see the crash logs from before the restart
kubectl get events -n prod --sort-by=.lastTimestamp # check for node pressure, scheduling failures, etc.
```

22. Monitoring & Observability Tools for Pods (Rancher, Prometheus, Grafana, Lens, K9s, Datadog)

22.1 Why You Need Dedicated Monitoring Tools

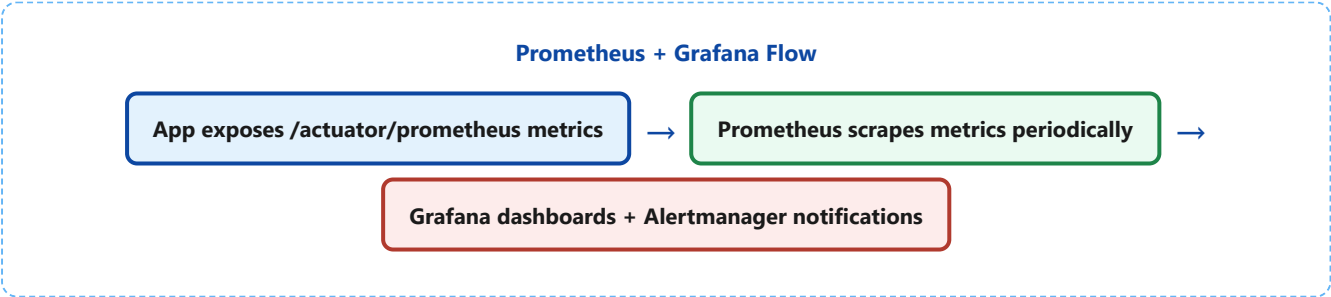
Running `kubectl get pods` repeatedly does not scale for real production clusters with hundreds of pods across multiple namespaces. Organizations use dedicated tools for a visual, historical, and alerting-capable view of cluster and application health.

22.2 Rancher — Cluster Management UI

Rancher provides a web UI to manage multiple Kubernetes clusters (on-prem, cloud, hybrid) from one place — view pods/deployments/logs, manage RBAC users, install apps from its catalog, and get a unified view across clusters that might otherwise use different cloud consoles (AKS, EKS, GKE, on-prem RKE).

- **Best for:** organizations managing multiple clusters across different clouds/on-prem, wanting a single pane of glass with built-in RBAC and app catalog.

22.3 Prometheus + Grafana — The Industry-Standard Metrics Stack



- **Prometheus:** a time-series database that scrapes metrics endpoints (Spring Boot Actuator exposes one via Micrometer) and stores them.
- **Grafana:** visualizes Prometheus data as dashboards — CPU/memory per pod, request latency, error rates, custom business metrics.
- **Alertmanager:** fires alerts (Slack, email, PagerDuty) based on Prometheus alert rules — e.g., "error rate > 5% for 5 minutes."

22.4 Lens & K9s — Developer-Focused Cluster Tools

Tool	Type	What It's Good For
Lens	Desktop GUI IDE for Kubernetes	Visual pod/resource browsing, built-in metrics, editing YAML live, great for day-to-day developer use
K9s	Terminal UI (TUI)	Extremely fast keyboard-driven navigation of pods/logs/exec without leaving the terminal — popular with SREs/DevOps

22.5 Datadog / New Relic / Dynatrace — Full Observability SaaS Platforms

Commercial APM (Application Performance Monitoring) platforms that combine metrics, distributed tracing, log aggregation, and infrastructure monitoring in one paid SaaS product — popular in larger enterprises that want a fully managed observability stack instead of running/maintaining Prometheus/Grafana themselves.

22.6 ELK / EFK Stack — Centralized Logging

Elasticsearch + Logstash/Fluentd + Kibana (or the Loki + Grafana alternative) aggregate logs from every pod across the cluster into one searchable place — essential since pods are ephemeral and their local logs disappear when they're deleted/rescheduled.

22.7 Real Organization Monitoring Stack Comparison

Tool	Category	Real-World Fit
Rancher	Cluster management UI	Multi-cluster, hybrid/on-prem heavy orgs
Prometheus + Grafana	Metrics + dashboards (self-hosted, open source)	Most common default in Kubernetes-native shops
Lens / K9s	Developer daily-driver tools	Individual developer/SRE productivity, not a replacement for team dashboards
Datadog/New Relic/Dynatrace	Full observability SaaS	Enterprises preferring managed tooling over self-hosting, willing to pay for it
ELK/EFK/Loki	Centralized logging	Any org needing searchable historical logs across ephemeral pods
Azure Monitor / Container Insights	Native Azure monitoring for AKS	Azure-native shops — integrates directly with AKS with minimal setup

Real scenario: "Our platform team moved from Rancher-only visibility to Rancher (cluster/RBAC management) + Prometheus/Grafana (metrics) + Loki (logs) + Alertmanager wired into our on-call Slack channel — this combination gave us both the management UI and the deep, queryable observability Rancher alone didn't provide."

23. Kubernetes Autoscaling & Self-Healing

23.1 Self-Healing Mechanisms

Failure	How K8s Self-Heals
Container crashes	kubelet restarts it per the pod's restart policy
Pod fails readiness probe	Removed from Service load-balancing until it passes again — no traffic sent to unhealthy pods
Pod fails liveness probe repeatedly	Kubelet kills and restarts the container
Node goes down	Controller Manager notices missed heartbeats, reschedules its pods onto healthy nodes
Fewer pods than desired replicas	ReplicaSet controller creates new pods immediately

23.2 Horizontal Pod Autoscaler (HPA)

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: order-service-hpa, namespace: prod }
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: order-service }
  minReplicas: 4
  maxReplicas: 20
  metrics:
  - type: Resource
    resource:
      name: cpu
      target: { type: Utilization, averageUtilization: 70 }
```

HPA watches a metric (CPU by default, or custom metrics via Prometheus Adapter — e.g., queue length, requests/sec) and automatically adds/removes pod replicas to keep it near the target.

23.3 Cluster Autoscaler / Node Autoscaling

While HPA scales **pods**, the Cluster Autoscaler (or AKS's built-in node autoscaling) scales the actual number of **worker nodes** — if pods can't be scheduled due to insufficient node capacity, it provisions new VM nodes automatically, and scales down idle nodes to save cost.

23.4 Vertical Pod Autoscaler (VPA)

Instead of adding more pod replicas, VPA adjusts the CPU/memory **requests/limits** of existing pods based on observed usage — less commonly used in production for latency-sensitive apps since it requires pod restarts to apply new resource values.

24. Kubernetes Security — Before K8s vs After K8s

24.1 Security Before Kubernetes (Plain Docker on Servers)

- No standard RBAC across many hosts — access control was per-server SSH keys/users.
- No built-in network policy — any container could often reach any other container/service on the host network.
- Secrets scattered across server config files, scripts, and shared drives.
- Manual, inconsistent patching per host.

24.2 What Kubernetes Adds for Security

Feature	Purpose
RBAC (Role-Based Access Control)	Fine-grained control over who can do what, per namespace/resource (e.g., QA team can only view pods in the qa namespace)
Network Policies	Firewall-like rules restricting which pods can talk to which — default is "all pods can talk to all pods" unless policies are applied
Pod Security Standards / Admission Controllers	Blocks pods from running as root, using host networking, or mounting sensitive host paths, unless explicitly allowed
Secrets + external secret providers	Centralizes and (with add-ons) encrypts sensitive config, integrates with Key Vault/Vault
Service Accounts	Identity for pods/apps to authenticate to the Kubernetes API or cloud services (e.g., AKS pod identity/workload identity to access Azure resources without embedded credentials)
Image Pull Secrets + private registries	Ensures only authorized images from trusted registries can be deployed
etcd encryption at rest	Protects the cluster's own state store (which contains Secrets) from disk-level exposure

24.3 Example RBAC Role (Real Pattern)

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: { namespace: qa, name: qa-read-only }
rules:
- apiGroups: [""]
  resources: ["pods", "services", "deployments"]
  verbs: ["get", "list", "watch"]      # read-only — QA team can view but not modify
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: { name: qa-team-binding, namespace: qa }
subjects:
- kind: Group
  name: "qa-team"
roleRef: { kind: Role, name: qa-read-only, apiGroup: rbac.authorization.k8s.io }
```

24.4 Example Network Policy (Zero-Trust Style)

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: order-service-allow, namespace: prod }
spec:
  podSelector: { matchLabels: { app: order-service } }
  policyTypes: [ Ingress ]
  ingress:
    - from:
      - podSelector: { matchLabels: { app: customer-portal } }    # only frontend can call order-service
      ports: [{ port: 8080 }]
```

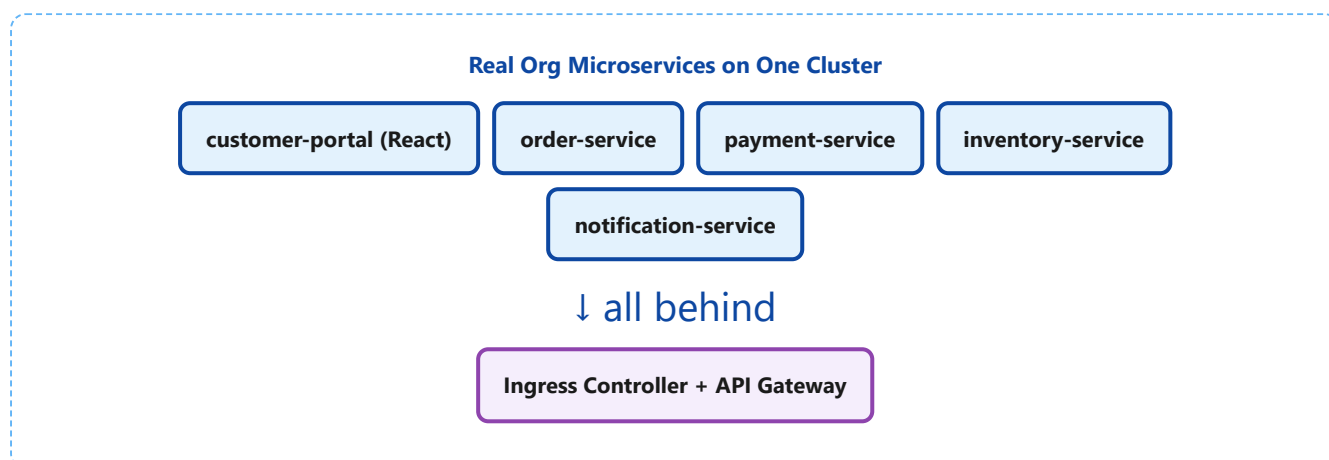
Interview tip: By default, Kubernetes networking is "flat" — any pod can reach any other pod. Network Policies let you move to a zero-trust model where services must be explicitly allowed to talk to each other, drastically reducing lateral movement if one service is compromised.

25. Common Kubernetes Issues & Real-Time Fixes

Issue	Cause	Real Fix
<code>ImagePullBackOff</code>	Wrong image tag/registry, or missing/expired <code>imagePullSecret</code>	Verify the tag exists in the registry; check <code>imagePullSecrets</code> and registry credentials
<code>CrashLoopBackOff</code>	App crashes on startup (bad config, missing env var, DB unreachable)	<code>kubectl logs --previous</code> to see the crash reason; fix config/secret and redeploy
Pod stuck in <code>Pending</code>	Insufficient node CPU/memory capacity, or unsatisfied node affinity/taints	<code>kubectl describe pod</code> to see scheduling failure reason; scale nodes or adjust resource requests
Readiness probe failing, pod never gets traffic	Probe path wrong, or app takes longer to start than <code>initialDelaySeconds</code> allows	Fix the health endpoint path, increase <code>initialDelaySeconds</code> / <code>timeoutSeconds</code>
Rolling update stuck halfway	New pods failing readiness probe, so old pods are kept (by design) to avoid downtime	Investigate why new version fails readiness; rollback with <code>kubectl rollout undo</code> if urgent
Service not routing to any pod	Service <code>selector</code> labels don't match pod labels	Confirm labels match exactly with <code>kubectl get pods -show-labels</code>
DNS resolution failing between services	CoreDNS pod issue, or wrong service name/namespace referenced	Use fully qualified name <code>service.namespace.svc.cluster.local</code> ; check CoreDNS pods are healthy
OOMKilled pods	Memory limit set too low for actual usage	Profile real memory usage, raise <code>resources.limits.memory</code> , fix memory leaks
Secrets exposed to too many users	Overly broad RBAC roles, or secrets not namespace-scoped properly	Apply least-privilege RBAC, split secrets per namespace/environment
Cluster running out of resources during traffic spike	No autoscaling configured, or max node count too low	Configure HPA + Cluster Autoscaler with realistic min/max limits, load test beforehand
"It works in one namespace but not another"	Missing ConfigMap/Secret/NetworkPolicy in the new namespace	Ensure per-namespace manifests (config, secrets, policies) are deployed consistently, ideally via Helm/Kustomize templates
<code>kubectl apply</code> succeeds but nothing actually changes	YAML applied to the wrong namespace/context, or a stale cached manifest reapplied	Always verify current context (<code>kubectl config current-context</code>) and target namespace explicitly with <code>-n</code> before applying
Ingress returns 502/504 for a healthy-looking service	Ingress Controller misconfigured backend path/port, or Service selector mismatch	Check Ingress Controller logs, confirm Service endpoints list actual pod IPs with <code>kubectl get endpoints</code>

26. Microservices on Kubernetes — Real Organization Setup

26.1 Typical Real Microservices Landscape



26.2 Service-to-Service Communication Patterns

- **Synchronous (REST/gRPC):** order-service calls payment-service directly via its ClusterIP Service DNS name — simple but couples availability.
- **Asynchronous (messaging):** order-service publishes an "OrderCreated" event to Kafka/RabbitMQ; inventory-service and notification-service consume independently — more resilient, decoupled.
- **Service Mesh (Istio/Linkerd):** adds mTLS between services, fine-grained traffic splitting (for Canary), and detailed service-to-service observability without changing app code.

26.3 Helm — Packaging Microservices for Kubernetes

Helm is the de-facto package manager for Kubernetes — a "Helm chart" templates your YAML (Deployment/Service/Ingress/ConfigMap) so the same chart can be reused across order-service, payment-service, etc., with only values (image tag, replica count, resource limits) differing per service/environment via a `values.yaml` file.

```

helm install order-service ./charts/microservice -f values-prod.yaml -n prod
helm upgrade order-service ./charts/microservice --set image.tag=483 -n prod
helm rollback order-service 1 -n prod
  
```

26.4 Shared Platform Concerns for Microservices

Concern	Real Solution
Centralized logging across services	EFK/Loki stack aggregating all pod logs
Distributed tracing (a request spanning multiple services)	OpenTelemetry + Jaeger/Zipkin, correlating trace IDs across service calls
Config consistency	Shared Helm chart + per-environment values files, or a config-server pattern

Service discovery	Built into Kubernetes DNS — no separate service registry (like Eureka) strictly needed on K8s
API Gateway concerns (auth, rate limiting)	Dedicated gateway (Kong, Azure API Management, or Spring Cloud Gateway) in front of all services

27. Azure Integration — AKS, ACR and Azure DevOps Pipeline

27.1 The Azure-Native Service Mapping

Generic Concept	Azure Service
Kubernetes cluster	Azure Kubernetes Service (AKS) — managed control plane, you manage node pools
Container registry	Azure Container Registry (ACR)
Secrets management	Azure Key Vault (via Secrets Store CSI Driver into pods)
Cluster monitoring	Azure Monitor / Container Insights
Load balancer / Ingress	Azure Load Balancer, or Application Gateway Ingress Controller (AGIC)
Pod-to-Azure-resource identity	Azure AD Workload Identity (replaces embedding service principal credentials in pods)

27.2 Real Azure DevOps Pipeline — Docker Build → AKS Deploy

```

trigger: { branches: { include: [ main ] } }
pool: { vmImage: 'ubuntu-latest' }

variables:
  - group: prod-secrets

stages:
- stage: BuildAndPush
  jobs:
  - job: Docker
    steps:
    - task: Docker@2
      inputs:
        containerRegistry: 'ACR-ServiceConnection'
        repository: 'order-service'
        command: 'buildAndPush'
        Dockerfile: 'Dockerfile'
        tags: '$(Build.BuildId)'
    - script: |
        trivy image --severity HIGH,CRITICAL --exit-code 1 \
          myacr.azurecr.io/order-service:$(Build.BuildId)
        displayName: 'Trivy Security Scan (blocks on HIGH/CRITICAL)'

- stage: DeployAKS
  dependsOn: BuildAndPush
  jobs:
  - deployment: DeployToAKS
    environment: 'Production' # approval gate configured on this Environment
    strategy:
      runOnce:
        deploy:
          steps:
          - task: KubernetesManifest@1
            inputs:
              action: 'deploy'
              kubernetesServiceConnection: 'AKS-Prod-ServiceConnection'
              namespace: 'prod'
              manifests: |
                k8s/deployment.yml
                k8s/service.yml
                k8s/ingress.yml
              containers: 'myacr.azurecr.io/order-service:$(Build.BuildId)'

```

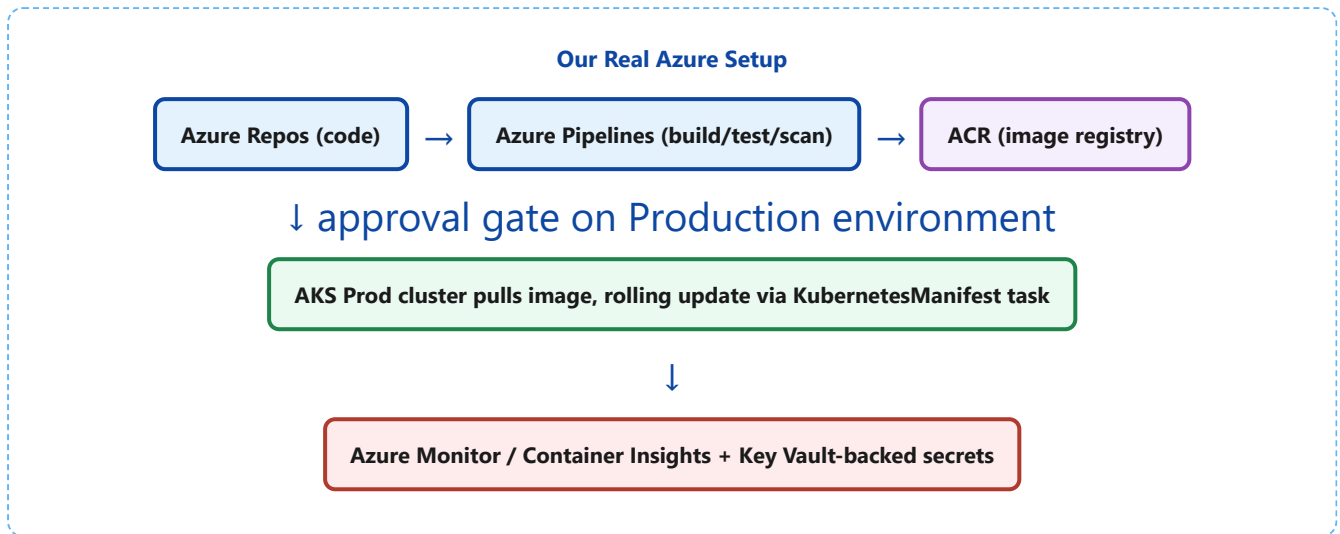
27.3 AKS Cluster Setup Basics (Real Steps)

```

az group create --name myapp-rg --location eastus
az aks create --resource-group myapp-rg --name myapp-aks \
  --node-count 3 --enable-cluster-autoscaler --min-count 3 --max-count 10 \
  --attach-acr myacr
az aks get-credentials --resource-group myapp-rg --name myapp-aks # configures local kubectl context
kubectl get nodes # verify cluster access

```

27.4 How Azure DevOps + AKS + ACR Fit Together End to End



28. End-to-End Real-Time Implementation Flow (Code → Prod)

1. Developer writes Spring Boot/React code locally, tests using `docker compose up` against local Postgres/Redis containers.
2. Push to feature branch → PR opened in Azure Repos with linked work item.
3. PR triggers CI: unit tests, integration tests (Testcontainers), SonarQube quality gate.
4. PR merged (2 reviewers) → full pipeline runs: Maven/npm build → Docker multi-stage build → Trivy security scan → push to ACR tagged with build ID.
5. Auto-deploy to Dev namespace in AKS → smoke tests run against Dev.
6. Auto-deploy to QA namespace → QA team runs manual + automated regression.
7. Manual approval (QA Lead) → deploy to Staging namespace → business/UAT sign-off.
8. Manual approval (Release Manager) → deploy to Production namespace using Rolling strategy with readiness/liveness probes gating traffic cut-over.
9. Post-deploy: Azure Monitor/Prometheus dashboards watched for error-rate/latency spikes; HPA reacts to real traffic; on-call alerted via Alertmanager/PagerDuty if thresholds are breached.
10. If something goes wrong: `kubectl rollout undo` or re-run pipeline with the previous build's image tag — production restored in minutes.

29. Docker vs Kubernetes vs Other Tools — Comparison

Tool	Category	What It Solves
Docker	Container runtime/packaging	Package an app + dependencies into one portable, isolated unit
Docker Compose	Local multi-container orchestration	Run a small multi-service stack on one machine easily
Kubernetes	Container orchestration at scale	Scheduling, scaling, self-healing, rollouts across many machines
Docker Swarm	Simpler built-in orchestrator	Lightweight orchestration; largely surpassed by Kubernetes' ecosystem in most orgs
Helm	Package manager for K8s	Templating and versioning of Kubernetes YAML across services/environments
Rancher	Cluster management UI	Multi-cluster visibility and RBAC across cloud/on-prem clusters
Istio/Linkerd	Service mesh	mTLS, traffic shaping, deep service-to-service observability
ArgoCD/Flux	GitOps continuous delivery	Cluster state is continuously reconciled from a Git repo, not just a one-time pipeline push

Interview tip: Docker and Kubernetes are not competitors — Kubernetes commonly **runs** containers built by Docker (or built via any OCI-compliant tool); the question "Docker vs Kubernetes" is really "packaging tool vs orchestration platform," and most real systems use both together.

30. Interview Questions & Answers (Basic → Advanced → Scenario Based)

Q1. Why did Docker replace/reduce reliance on Virtual Machines?

Docker containers share the host OS kernel instead of bundling a full guest OS per workload, making them far lighter (MBs vs GBs), faster to start (seconds vs minutes), and more portable — the same image runs identically on any host with a compatible container runtime.

Q2. What is the difference between an image and a container?

An image is an immutable, read-only build template made of layers. A container is a running (or stopped) instance of that image with an added writable layer for runtime changes. One image can produce many running containers.

Q3. Explain multi-stage Docker builds and why they matter.

Multi-stage builds let you use one Dockerfile with several `FROM ... AS stage` blocks — a heavy "build" stage compiles the app, and a minimal "runtime" stage copies only the final artifact. This keeps the shipped image small and avoids bundling compilers/build tools that increase size and attack surface.

Q4. How many images can one Dockerfile produce?

Unlimited — a Dockerfile is just a recipe. Every `docker build` run (with different tags, build args, or source code state) produces a distinct image. Real orgs commonly tag by build number, git SHA, and semantic version for the same underlying Dockerfile.

Q5. What's the difference between CMD and ENTRYPOINT?

ENTRYPOINT defines the fixed command that always runs and is hard to override; CMD provides default arguments that can be easily overridden at `docker run`. The common real-world pattern combines both — ENTRYPOINT for the binary, CMD for overridable default args.

Q6. How do you keep Docker images secure?

Use minimal base images (Alpine/Distroless), run as a non-root user, use multi-stage builds so build tools don't ship, scan images for CVEs (Trivy/Snyk) in the pipeline, never bake secrets into layers, and pull only from trusted private registries.

Q7. Why do we need Kubernetes if we already have Docker?

Docker runs a single container on a single host well, but doesn't handle multi-host scheduling, automatic scaling, self-healing after crashes/node failures, zero-downtime rollouts, or service discovery across many machines — Kubernetes orchestrates all of that declaratively.

Q8. Explain the relationship between Pod, ReplicaSet, and Deployment.

A Pod is the smallest deployable unit (one or more containers). A ReplicaSet ensures a set number of identical Pods are always running. A Deployment manages ReplicaSets and adds rollout/rollback strategy on top — in practice, you almost always work with Deployments directly.

Q9. What's the difference between a readiness probe and a liveness probe?

A readiness probe determines if a pod is ready to receive traffic — if it fails, the pod is removed from the Service's endpoints but not restarted. A liveness probe determines if a pod is alive at all — if it fails repeatedly, kubelet kills and restarts the container.

Real scenario: A Spring Boot service that's still warming up its cache on startup should fail readiness (so it gets no traffic yet) without failing liveness (so it isn't needlessly restarted while legitimately starting up).

Q10. How does a rolling update actually achieve zero downtime?

The Deployment creates new pods gradually (controlled by `maxSurge`) while removing old pods gradually (controlled by `maxUnavailable`), and only routes traffic to new pods once they pass their readiness probe — so there's always a sufficient number of healthy pods serving traffic throughout the rollout.

Q11. What tools do you use for monitoring pods in production, and why more than one?

Rancher for multi-cluster management/RBAC visibility, Prometheus + Grafana for metrics and dashboards, Loki/EFK for centralized logs, and Alertmanager for on-call alerting. Each tool covers a different observability pillar (management, metrics, logs, alerts) — no single tool covers all of them well.

Q12. How do Secrets differ from ConfigMaps, and are Kubernetes Secrets actually secure by default?

ConfigMaps hold non-sensitive config; Secrets hold sensitive values but are only base64-encoded by default — not encrypted. Real production setups add etcd encryption-at-rest and/or integrate an external secret manager like Azure Key Vault via the Secrets Store CSI Driver for genuine encryption and centralized rotation.

Q13. A pod is stuck in CrashLoopBackOff — walk through your debugging steps.

First `kubectl describe pod` to see recent events and exit code; then `kubectl logs --previous` to see the crash's actual error output; check for missing/incorrect ConfigMap or Secret values, a misconfigured health check path, or an unreachable dependency (DB/downstream service); fix and reapply, watching `kubectl rollout status`.

Q14. How would you roll back a bad production deployment in Kubernetes?

`kubectl rollout undo deployment/order-service -n prod` instantly reverts to the previous ReplicaSet revision — no rebuild needed since the previous image is still in the registry and the previous ReplicaSet spec is retained in rollout history.

Q15. How does Kubernetes autoscaling work end to end during a traffic spike?

The Horizontal Pod Autoscaler watches a metric (e.g., CPU utilization) and increases pod replicas when it exceeds the target. If the cluster's existing nodes can't fit the new pods, the Cluster Autoscaler (or AKS node autoscaling) provisions additional worker nodes automatically, and scales back down once demand drops.

Q16. How do you integrate Docker/Kubernetes into an Azure DevOps pipeline?

A Docker@2 task builds and pushes the image to Azure Container Registry using a Service Connection; a subsequent deployment job targeting a Kubernetes-backed Environment (with approval gates) uses the KubernetesManifest@1 task with an AKS Service Connection to apply the Deployment/Service/Ingress manifests, substituting the new image tag.

Q17. What real-world issue have you faced with Docker/Kubernetes and how did you fix it?

Example: a Spring Boot service was repeatedly OOMKilled in production despite "normal-looking" heap usage — root cause was the JVM not respecting the container's cgroup memory limit on an older JDK image. Fixed by upgrading the base image and explicitly setting `-XX:MaxRAMPercentage`, aligned with the pod's configured memory limit.

Q18. What is the difference between a Service and an Ingress?

A Service provides a stable internal (or basic external via LoadBalancer/NodePort) network endpoint for a set of pods. An Ingress sits in front of Services and handles external HTTP/HTTPS routing by host/path plus TLS termination, typically via a single shared Ingress Controller for the whole cluster instead of one LoadBalancer per service.

Q19. How do microservices communicate on Kubernetes, and what's the tradeoff between sync and async calls?

Synchronous REST/gRPC calls via Service DNS names are simple but couple the caller's availability to the callee's uptime. Asynchronous messaging (Kafka/RabbitMQ) decouples services — the publisher doesn't wait on the consumer — improving resilience at the cost of added infrastructure and eventual-consistency complexity.

Q20. What's the difference between Docker Swarm and Kubernetes?

Both orchestrate containers, but Kubernetes has a far larger ecosystem, more granular objects (ConfigMaps, RBAC, NetworkPolicies, CRDs), and is the de-facto industry standard; Swarm is simpler to set up but has largely fallen out of favor in real organizations compared to Kubernetes (especially managed offerings like AKS/EKS/GKE).

31. Final Summary & Cheat Sheet

Before Docker: physical servers and VMs were heavy, slow to provision, and caused "works on my machine" inconsistency between Dev/QA/Prod.

After Docker: apps are packaged with their exact dependencies into small, fast, portable images that run identically everywhere.

Before Kubernetes: running many containers across many hosts required manual scaling, manual restarts, and manual networking glue.

After Kubernetes: you declare desired state (replica count, resources, health checks) and the cluster continuously self-heals, scales, and rolls out updates with zero downtime.

Our real Azure setup: Azure Repos → Azure Pipelines (build, test, Trivy scan) → Azure Container Registry → Azure Kubernetes Service, with approval gates on Staging/Production Environments, secrets from Azure Key Vault, and monitoring via Azure Monitor/Prometheus/Grafana.

31.1 Quick Cheat Sheet

Term	One-Line Meaning
Dockerfile	Text recipe describing how to build an image
Image	Immutable, layered, versioned build output
Container	A running instance of an image
Registry	Where images are stored/distributed (ACR/Docker Hub/ECR)
Multi-stage build	One Dockerfile, multiple stages, only final stage ships
Pod	Smallest deployable unit in Kubernetes
Deployment	Manages ReplicaSets, handles rollout/rollback strategy
Service	Stable internal network identity for a set of pods
Ingress	External HTTP routing + TLS termination into the cluster
ConfigMap/Secret	Externalized non-sensitive/sensitive configuration
Namespace	Logical isolation within one physical cluster
HPA	Automatically scales pod replicas based on metrics
Rolling Update	Default zero-downtime deployment strategy in Kubernetes
Rancher	Multi-cluster management UI
Prometheus/Grafana	Metrics collection + dashboards/alerts
Helm	Package manager / templating for Kubernetes YAML

Use this guide alongside the Git/GitHub/GitLab/Bitbucket guide and the CI/CD guide — together they cover the complete real-world Java full-stack developer journey: writing code, versioning it, automating build/test/deploy, and running it reliably in containers on Kubernetes in production.